

Цикли

Функція range()

Функція `range()` як універсальний інструмент генерування цілих чисел зазвичай використовується для організації циклів з відомою кількістю ітерацій. Вона може приймати один, два або три аргументи.

У випадку з одним аргументом функція генерує цілі числа від 0 до значення аргумента, не включаючи останнього:

```
range(5)          # генерує числа 0, 1, 2, 3, 4
```

Якщо функції передати два аргументи, перший вважатиметься початком відліку:

```
range(-3, 3)      # генерує числа -3, -2, -1, 0, 1, 2
```

Третім аргументом можна задавати крок:

```
range(-10, 10, 3) # генерує числа -10, -7, -4, -1, 2, 5, 8
```

Крок може бути від'ємним:

```
range(5, 0, -1)   # генерує числа 5, 4, 3, 2, 1
```

При цьому за некоректних значень цілочислових аргументів інтерпретатор не видасть повідомлення про помилку — просто не буде нічого згенеровано.

Цикл for

Цикл `for` зазвичай використовуються при відомій кількості повторень і має такий синтаксис¹:

```
for <ім'я_змінної> in range(<аргументи>):  
    <блок інструкцій>
```

¹ Конструкція `for...in` також часто застосовується для генерування списків, словників і множин, порядкового обходу файлів, посимвольного обходу рядків та поелементного обходу колекцій (розглядатиметься у наступних розділах)

Як і у випадку з if, for є складеною інструкцією — заголовок циклу закінчується двокрапкою, а інструкції тіла циклу оформляються з відступами. Змінна, яка використовується у заголовку циклу for, по чергово приймає значення, згенеровані функцією range(). У випадку функції range() з одним аргументом її називають лічильником — така назва стає зрозумілою з наступного коду:

```
for number in range(5):  
    print(number, end = ' ')      # 0 1 2 3 4
```

Наступний фрагмент коду демонструє застосування циклу for для обчислення суми та добутку перших n натуральних чисел:

```
n = int(input('Введіть n: '))  
suma, dobutok = 0, 1  
for i in range(1,n+1):  
    suma = suma + i  
    dobutok = dobutok * i  
print(suma, dobutok)
```

У Python прийнято використовувати так звані комбіновані присвоєння:

```
x += a      # аналог x = x + a  
x -= a      # аналог x = x - a  
x *= a      # аналог x = x * a  
x /= a      # аналог x = x / a  
x %= a      # аналог x = x % a  
...
```

Так, з їх використанням тіло циклу з попереднього коду може бути записане простіше:

```
for i in range(1,n+1):  
    suma += i  
    dobutok *= i
```

Цикл while

На відміну від циклу for, тіло якого виконується заздалегідь відому кількість разів, інструкція while дає змогу організувати цикл, кількість ітерацій якого непередбачувана. Найпростіший синтаксис цієї складеної інструкції такий:

```
while <логічний вираз>:  
    <блок іструкцій>                # виконується, поки логічний_вираз дорівнює True
```

Якщо постановка задачі вимагає завершення циклу (насправді існують і реалізації нескінченного циклу), то зрозуміло, що принаймі одна з інструкцій вкладеного блоку повинна змінювати значення деякої змінної, яка входить у логічний вираз і може надати йому значення False.

Наступний фрагмент коду демонструє застосування циклу while для генерування випадкових цифр, допоки не буде згенеровано 0:

```
from random import randint

number = randint(0,9)
while number != 0:                # можна використати скорочений запис while number:
    print(number)                 # друкуємо поточну цифру
    number = randint(0,9)         # генеруємо наступну цифру
```

Інструкція break

Інструкція break використовується для примусового виходу з циклу. Тобто, її виконання змушує інтерпретатор перейти до першої інструкції після циклу. Так, попередній код з використанням інструкції break може бути переписаний у вигляді, який не вимагає допоміжної інструкції перед циклом, але містить перевірку умови в тілі циклу:

```
from random import randint

while True:
    number = randint(0,9)
    if not number:
        break
    print(number)
```

Крім цього, в наведеному коді використано так званий нескінченний цикл: оскільки логічний вираз після інструкції while завжди дорівнює True, то тіло циклу виконувалося б нескінченно, якби не було інструкції break.

Інструкція continue

Інструкція continue, як і break, використовується виключно всередині циклу. Вона змушує інтерпретатор пропустити усі інструкції тіла циклу, що йдуть після неї і повернутись у заголовок циклу, тобто виконати примусовий перехід до наступної ітерації (якщо, звісно, умова у заголовку циклу виконається).

Наступний фрагмент коду демонструє застосування інструкції `continue` для генерування випадкових парних цифр, допоки не буде згенеровано 0:

```
from random import randint
number = 1 # будь-яка цифра для того, щоб увійти в цикл
while number:
    number = randint(0,9)
    if number % 2:
        continue
    print(number)
```

Блок `else`

Цикли мають і розширений синтаксис, який містить необов'язковий блок `else`, який виконується тільки у випадку непримусового завершення циклу, тобто, коли логічний вираз у заголовку циклу `while` повернув значення `False` або лічильник циклу `for` перебрав усі значення. Це означає, що блок `else` є зміст використовувати лише у випадках, коли тіло циклу містить інструкцію `break`, інакше він виконуватиметься завжди.

Розглянемо іструкцію `while/else` на прикладі програми для перевірки, чи введене ціле число є простим:

```
x = int(input('Введіть ціле число: '))
i = 2 # перший можливий дільник
while i <= x // 2: # дільник не може перевищувати половини числа
    if x % i == 0:
        print('Число не є простим')
        break # забезпечуємо вихід з циклу в обхід блоку else
    i += 1 # беремо наступний можливий дільник
else: # якщо інструкція break не виконається,
    print('Число просте')
```

Це саме можна реалізувати і з використанням циклу `for`:

```
x = int(input('Введіть ціле число: '))
for i in range(2, x//2 + 1):
    if x % i == 0:
        print('Число не є простим')
        break
else:
    print('Число просте')
```
